```
!nvidia-smi
```

```
Thu May  7 18:41:09 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   45C    P8             10W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
%%writefile vector_add.cu

#include <stdio.h>

__global__ void vectorAdd(int *A, int *B, int *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int n = 1024;

    int h_A[n], h_B[n], h_C[n];

    // Initialize vectors
    for (int i = 0; i < n; i++) {
        h_A[i] = i;
        h_B[i] = i * 2;
    }

    int *d_A, *d_B, *d_C;

    cudaMalloc((void**)&d_A, n * sizeof(int));
    cudaMalloc((void**)&d_B, n * sizeof(int));
    cudaMalloc((void**)&d_C, n * sizeof(int));

    cudaMemcpy(d_A, h_A, n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, n * sizeof(int), cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (n + blockSize - 1) / blockSize;

    vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);

    cudaMemcpy(h_C, d_C, n * sizeof(int), cudaMemcpyDeviceToHost);

    printf("First 10 Results:\\n");

    for (int i = 0; i < 10; i++) {
        printf("%d + %d = %d\\n", h_A[i], h_B[i], h_C[i]);
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

```
Writing vector_add.cu
```

```
!nvcc vector_add.cu -o vector_add
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future
```

```
!./vector_add
```

```
First 10 Results:\n0 + 0 = 0\n1 + 2 = 3\n2 + 4 = 6\n3 + 6 = 9\n4 + 8 = 12\n5 + 10 = 15\n6 + 12 = 18\n7 + 14 = 21\n8 + 16 = 2
```

Start coding or generate with AI.